

Manchester United — Premier League 2025/26

Análisis estadístico y comparación con la liga

Daniel Arellano

2026-08-16

Contenido

1 Fuentes de datos	1
1.1 Tabla maestra por equipo	2
2 El United en el contexto de la liga	4
3 Eficiencia: dónde se ganó y dónde se perdió la temporada	5
3.1 El coste de rematar mucho y mal	5
3.2 La portería como agujero	8
4 La temporada partido a partido	12
5 Plantilla completa	14
5.1 Reparto de minutos	14
5.2 Producción ajustada por minutos	16
5.3 Bruno Fernandes, otra vez el eje	17
6 Síntesis	18
7 Líneas de análisis abiertas	18

1. Fuentes de datos

Este análisis usa **FBref** (Sports Reference) como fuente principal, consultada el 16 de agosto de 2026. Una versión anterior del documento partía de la ficha de Marca.com; se cambió de fuente por dos motivos, y el contraste entre ambas está documentado en **?@sec-contraste** porque explica varias conclusiones que hubo que revisar.

Lo que aporta FBref y Marca no tenía:

- **Los 20 equipos de la liga**, con las mismas definiciones para todos, que es la condición para que cualquier comparación signifique algo.
- **La plantilla completa** (27 jugadores con minutos), en lugar de un top 3 por categoría.
- **Los 38 partidos uno a uno**, con rival, resultado, posesión, formación y asistencia.

- Métricas que Marca no publica: paradas, porterías a cero, minutos, centros, intercepciones, edad y titularidades.

```
clasificacion = pd.read_csv(DATOS / "clasificacion.csv")
std_favor      = pd.read_csv(DATOS / "standard_favor.csv")
std_contra     = pd.read_csv(DATOS / "standard_contra.csv")
tiros_favor    = pd.read_csv(DATOS / "tiros_favor.csv")
tiros_contra   = pd.read_csv(DATOS / "tiros_contra.csv")
portero        = pd.read_csv(DATOS / "portero_favor.csv")
misc_favor     = pd.read_csv(DATOS / "misc_favor.csv")
misc_contra    = pd.read_csv(DATOS / "misc_contra.csv")

united_plantilla = pd.read_csv(DATOS / "united_plantilla.csv")
united_misc      = pd.read_csv(DATOS / "united_misc.csv")
united_minutos   = pd.read_csv(DATOS / "united_minutos.csv")
united_partidos  = pd.read_csv(DATOS / "united_partidos.csv", parse_dates=["date"])
contraste_marca  = pd.read_csv(DATOS / "marca_united.csv")

# Las tablas "contra" traen el equipo como "vs Arsenal"; se normaliza para poder cruzar.
for df in (std_contra, tiros_contra, misc_contra):
    df["team"] = df["team"].str.removeprefix("vs ")

print(f"{clasificacion.shape[0]} equipos · "
      f"{united_plantilla.shape[0]} jugadores del United con minutos · "
      f"{(united_partidos['comp'] == 'Premier League').sum()} partidos de liga")
```

20 equipos · 27 jugadores del United con minutos · 38 partidos de liga

1.1. Tabla maestra por equipo

Se construye un único DataFrame por equipo cruzando las seis tablas. Todo el análisis comparado sale de aquí.

```
liga = (clasificacion[["rank", "team", "wins", "ties", "losses",
                      "goals_for", "goals_against", "goal_diff", "points"]]
        .merge(std_favor[["team", "possession", "avg_age", "players_used",
                          "assists", "pens_made", "pens_att"]], on="team")
        .merge(tiros_favor[["team", "shots", "shots_on_target", "shots_on_target_pct",
                           "goals_per_shot", "goals_per_shot_on_target"]], on="team")
        .merge(tiros_contra[["team", "shots", "shots_on_target", "shots_on_target_pct"]],
               on="team", suffixes=("", "_contra"))
        .merge(portero[["team", "gk_saves", "gk_save_pct", "gk_clean_sheets",
                        "gk_clean_sheets_pct"]], on="team")
        .merge(misc_favor[["team", "cards_yellow", "cards_red", "fouls", "fouled",
                           "offsides", "crosses", "interceptions", "tackles_won"]],
               on="team")
        )
```

```

# Métricas derivadas
liga["tiros_pp"] = liga["shots"] / PJ
liga["tiros_puerta_pp"] = liga["shots_on_target"] / PJ
liga["conversion"] = liga["goals_for"] / liga["shots"] * 100
liga["conv_puerta"] = liga["goals_for"] / liga["shots_on_target"] * 100
liga["gol_concedido_por_tiro_puerta"] = liga["goals_against"] / liga["shots_on_target_contra"] *
liga["puntos_pp"] = liga["points"] / PJ

liga = liga.sort_values("rank").reset_index(drop=True)

def rk(col, ascendente=False):
    """Puesto del United en la liga para una métrica (1 = mejor)."""
    orden = liga.sort_values(col, ascending=ascendente).reset_index(drop=True)
    return int(orden.index[orden["team"] == UTD][0]) + 1

utd = liga.loc[liga["team"] == UTD].iloc[0]
liga.shape

```

(20, 41)

Tabla 1: Premier League 2025/26 — tabla maestra por equipo.

Pos	Equipo	Pts	GF	GC	Dif	Pos. %	Tiros	A puerta	Paradas %	Vallas 0
1	Arsenal	85	71	27	44	56.4	552	186	69.7	19
2	Manchester City	78	77	35	42	60.7	595	205	72.2	16
3	Manchester Utd	71	69	50	19	51.9	595	215	64.8	8
4	Aston Villa	65	56	49	7	53.7	483	170	69.9	9
5	Liverpool	60	63	53	10	59.3	588	175	65.1	10
6	Bourne- mouth	57	58	54	4	50.1	526	178	66.9	11
7	Sunderland	54	42	48	-6	44.9	396	130	71.1	11
8	Brighton	53	52	46	6	53.9	494	176	69.5	10
9	Brentford	53	55	52	3	47.5	405	147	67.1	10
10	Chelsea	52	58	52	6	57.7	509	166	66.3	9
11	Fulham	52	47	51	-4	51.7	481	138	65.8	9
12	Newcastle	49	53	55	-2	52.5	492	176	67.3	8
13	Everton	49	47	50	-3	43.8	422	142	66.4	11
14	Leeds United	47	49	56	-7	45.7	477	151	64.3	8
15	Crystal Palace	45	41	51	-10	45.9	442	141	67.9	12
16	Nottingham	44	48	51	-3	46.8	476	156	67.7	9
17	Tottenham	41	48	57	-9	50.4	435	146	62.0	9

Pos	Equipo	Pts	GF	GC	Dif	Pos. %	Tiros	A puerta	Paradas %	Vallas 0
18	West Ham	39	46	65	-19	42.6	404	142	66.7	7
19	Burnley	22	38	75	-37	42.5	355	118	64.6	4
20	Wolves	20	27	68	-41	42.8	369	123	62.0	4

2. El United en el contexto de la liga

Tabla 2: Posición del United en la liga (1 = mejor de los 20).

Métrica	United	Puesto en la liga
Puntos	71.00	3
Goles a favor	69.00	3
Goles en contra	50.00	6
Diferencia de goles	19.00	3
Tiros totales	595.00	2
Tiros a puerta	215.00	1
Precisión de tiro (%)	36.10	2
Goles por tiro a puerta	0.29	9
Posesión (%)	51.90	8
Tiros a puerta concedidos	141.00	3
Porcentaje de paradas	64.80	16
Porterías a cero	8.00	15
Entradas ganadas	406.00	5
Faltas cometidas	390.00	6

Dos cosas saltan de esa tabla y tiran en direcciones opuestas.

El United fue el equipo que más veces tiró a puerta de toda la Premier League: 215, por delante del City (205) y del Arsenal (186). Empató con el City en tiros totales (595) y tuvo la mejor precisión de disparo de la liga (36,1 %). En generación de peligro no fue el tercero: fue el primero.

Y a la vez tuvo el quinto peor porcentaje de paradas de la liga (64,8 %, puesto 16 de 20), por delante sólo de Wolves, Tottenham, Leeds y Burnley —tres descendidos o en zona baja—. Con 8 porterías a cero, decimoséptimo de veinte, pese a conceder muy pocos tiros a puerta.

```
d = liga.sort_values("shots_on_target")
fig, ax = plt.subplots(figsize=(7, 5))
ax.barh(d["team"], d["shots_on_target"], color=resaltar(d["team"]))
ax.set_xlabel("Tiros a puerta en la temporada")
ax.set_title("Tiros a puerta - Premier League 2025/26")
ax.grid(axis="y", visible=False)
for i, (_, r) in enumerate(d.iterrows()):
    ax.annotate(f"{int(r['shots_on_target'])}", (r["shots_on_target"], i),
                xytext=(3, 0), textcoords="offset points", va="center", fontsize=7.5)
```

```
fig.tight_layout()
plt.show()
```

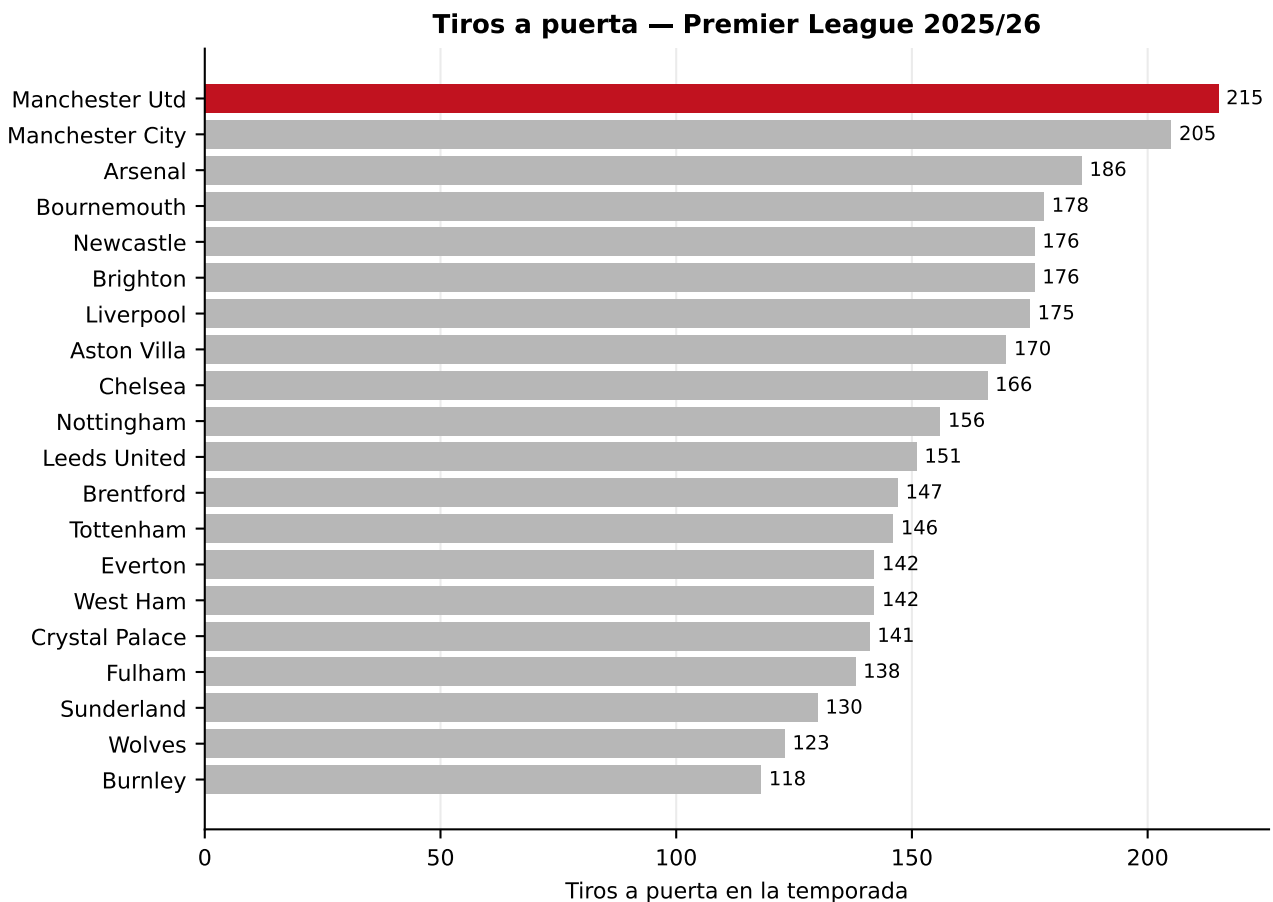


Figura 1: Tiros a puerta por equipo. El United lidera la liga en generación de remate franco.

3. Eficiencia: dónde se ganó y dónde se perdió la temporada

3.1. El coste de rematar mucho y mal

Si el United lideró la liga en tiros a puerta pero terminó tercero en goles, la diferencia está en la conversión. Y ahí el dato es incómodo.

```
fig, ax = plt.subplots(figsize=(8, 5.4))
ax.scatter(liga["shots_on_target"], liga["goals_for"],
          s=55, color=[ROJO if t == UTD else AZUL for t in liga["team"]],
          zorder=3, edgecolor="white", linewidth=0.8)

# Recta de conversión media de la liga
tasa_media = liga["goals_for"].sum() / liga["shots_on_target"].sum()
xs = np.array([liga["shots_on_target"].min(), liga["shots_on_target"].max()])
```

```

ax.plot(xs, xs * tasa_media, "--", color=GRIS, linewidth=1.2, zorder=1,
        label=f"Conversión media de la liga ({tasa_media*100:.1f}%)")

# Se etiquetan los 20 equipos; el United primero para asegurarle sitio, y el
# resto de mayor a menor distancia respecto a la recta media, que es lo que
# hace interesante a un punto en este gráfico.
d = liga.assign(desvio=(liga["goals_for"] - liga["shots_on_target"] * tasa_media).abs())
d = pd.concat([d[d["team"] == UTD], d[d["team"] != UTD].sort_values("desvio", ascending=False)])

etiquetar(ax, d["shots_on_target"].values, d["goals_for"].values,
          [CORTO.get(t, t) for t in d["team"]],
          colores=[ROJO if t == UTD else "#333333" for t in d["team"]],
          negritas=[t == UTD for t in d["team"]])

ax.set_xlabel("Tiros a puerta")
ax.set_ylabel("Goles marcados")
ax.set_title("Volumen de remate frente a goles")
ax.margins(x=0.10, y=0.10)
ax.legend(frameon=False, loc="upper left")
fig.tight_layout()
plt.show()

```

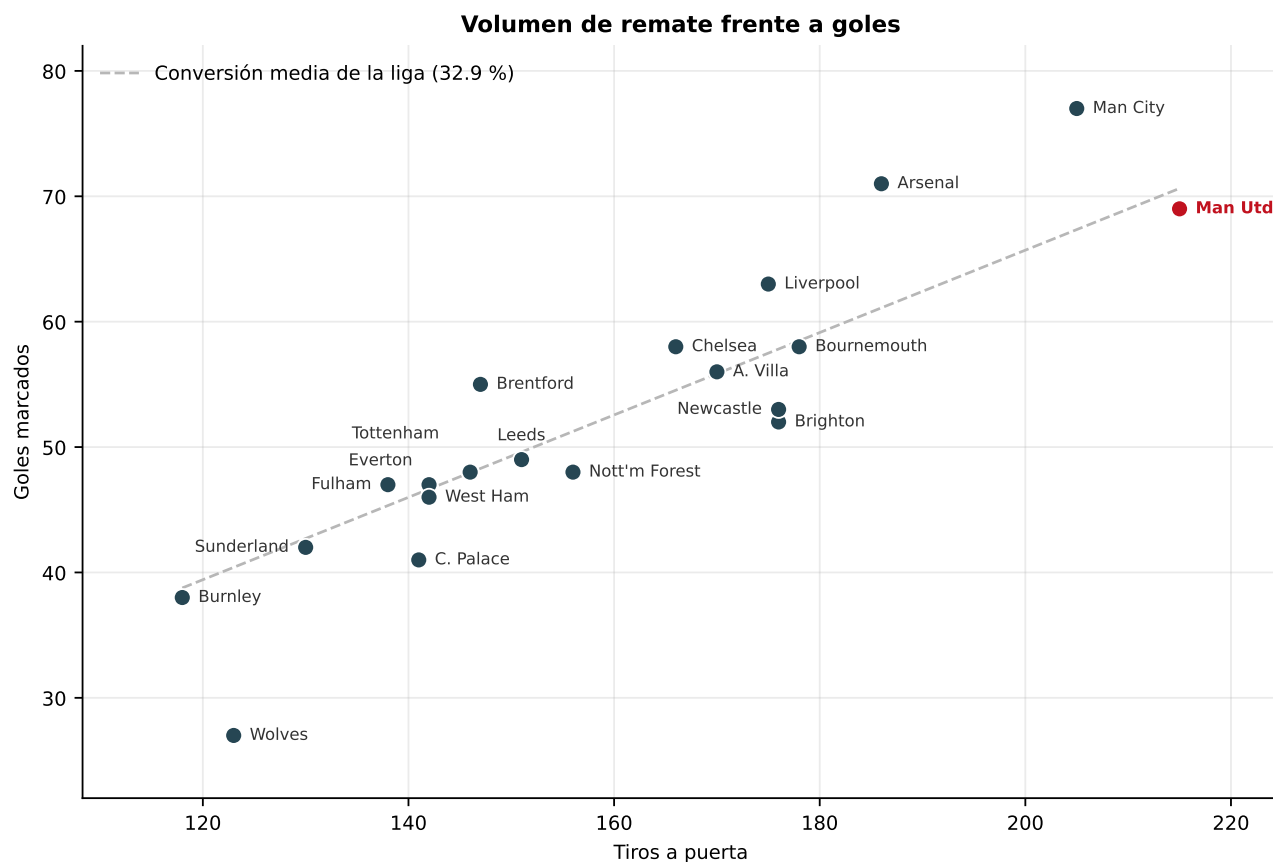


Figura 2: Tiros a puerta contra goles marcados. La línea es el rendimiento medio de la liga; por debajo de ella se convierte peor de lo esperado.

Tabla 3: Conversión de tiros a puerta, los 20 equipos. La última columna es la métrica que publica FBref, que excluye penaltis; la penúltima incluye todos los goles.

#	Equipo	Tiros a puerta	Goles	Conversión %	G sin penalti / tiro a puerta
1	Arsenal	186	71	38.17	0.34
2	Manchester City	205	77	37.56	0.35
3	Brentford	147	55	37.41	0.31
4	Liverpool	175	63	36.00	0.34
5	Chelsea	166	58	34.94	0.30
6	Fulham	138	47	34.06	0.28
7	Everton	142	47	33.10	0.30
8	Aston Villa	170	56	32.94	0.31
9	Tottenham	146	48	32.88	0.32
10	Bournemouth	178	58	32.58	0.29
11	Leeds United	151	49	32.45	0.28
12	West Ham	142	46	32.39	0.29
13	Sunderland	130	42	32.31	0.26
14	Burnley	118	38	32.20	0.28

#	Equipo	Tiros a puerta	Goles	Conversión %	G sin penalti / tiro a puerta
15	Manchester Utd	215	69	32.09	0.29
16	Nottingham	156	48	30.77	0.28
17	Newcastle	176	53	30.11	0.27
18	Brighton	176	52	29.55	0.27
19	Crystal Palace	141	41	29.08	0.23
20	Wolves	123	27	21.95	0.19

⚠ Dos definiciones de gol conviven en estos datos

FBref publica **66 goles** del United y la clasificación, **69**. No es un error: la tabla de plantilla suma sólo los goles anotados por jugadores del equipo, mientras que la clasificación incluye los **3 goles en propia puerta a favor**. Además, `goals_per_shot_on_target` **excluye los penaltis**: los 0,29 del United salen de $(66 - 4) \div 215$.

En este documento se usa el criterio de la clasificación —69 goles, todo incluido— para cualquier cálculo propio, y la columna de FBref se conserva etiquetada como lo que es. Mezclar ambas en una misma comparación es el error fácil aquí.

```
goles_esperados = utd["shots_on_target"] * tasa_media
brecha = utd["goals_for"] - goles_esperados
print(f"Con {int(utd['shots_on_target'])} tiros a puerta y la conversión media de "
      f"la liga ({tasa_media*100:.1f}%), al United le habrían correspondido "
      f"**{goles_esperados:.0f} goles**. Marcó **{int(utd['goals_for'])}**: "
      f"**{brecha:+.0f}** respecto a lo esperado por volumen.\n\n"
      f"El City marcó {int(liga.loc[liga['team']=='Manchester City','goals_for'].iloc[0])} "
      f"con {int(liga.loc[liga['team']=='Manchester City','shots_on_target'].iloc[0])} "
      f"tiros a puerta; el Arsenal, "
      f"{int(liga.loc[liga['team']=='Arsenal','goals_for'].iloc[0])} con "
      f"{int(liga.loc[liga['team']=='Arsenal','shots_on_target'].iloc[0])}.")
```

Con 215 tiros a puerta y la conversión media de la liga (32.9%), al United le habrían correspondido **71 goles**. Marcó **69: -2** respecto a lo esperado por volumen.

El City marcó 77 con 205 tiros a puerta; el Arsenal, 71 con 186.

Con la conversión del Arsenal (38.2% en vez del 32.1% del United) esos 215 tiros a puerta habrían dado **82 goles** en lugar de 69. Con la del City, **81**.

Ésta es la lectura central de la temporada: **el United generó más remate franco que nadie y lo aprovechó por debajo de la media de la liga**. No es un problema de creación, es de definición: el margen está en quién remata y desde dónde, no en llegar más veces.

3.2. La portería como agujero


```

fig, ax = plt.subplots(figsize=(8, 5.4))
ax.scatter(liga["shots_on_target_contra"], liga["goals_against"],
           s=55, color=[ROJO if t == UTD else AZUL for t in liga["team"]],
           zorder=3, edgecolor="white", linewidth=0.8)

tasa_def = liga["goals_against"].sum() / liga["shots_on_target_contra"].sum()
xs = np.array([liga["shots_on_target_contra"].min(), liga["shots_on_target_contra"].max()])
ax.plot(xs, xs * tasa_def, "--", color=GRIS, linewidth=1.2, zorder=1,
        label=f"Media de la liga ({tasa_def*100:.1f}% de los tiros a puerta acaban en gol)")

d = liga.assign(desvio=(liga["goals_against"] - liga["shots_on_target_contra"] * tasa_def).abs())
d = pd.concat([d[d["team"] == UTD], d[d["team"] != UTD].sort_values("desvio", ascending=False)])

etiquetar(ax, d["shots_on_target_contra"].values, d["goals_against"].values,
           [CORTO.get(t, t) for t in d["team"]],
           colores=[ROJO if t == UTD else "#333333" for t in d["team"]],
           negritas=[t == UTD for t in d["team"]])

ax.set_xlabel("Tiros a puerta concedidos")
ax.set_ylabel("Goles encajados")
ax.set_title("Volumen concedido frente a goles encajados")
ax.margins(x=0.10, y=0.10)
ax.legend(frameon=False, loc="upper left")
fig.tight_layout()
plt.show()

```

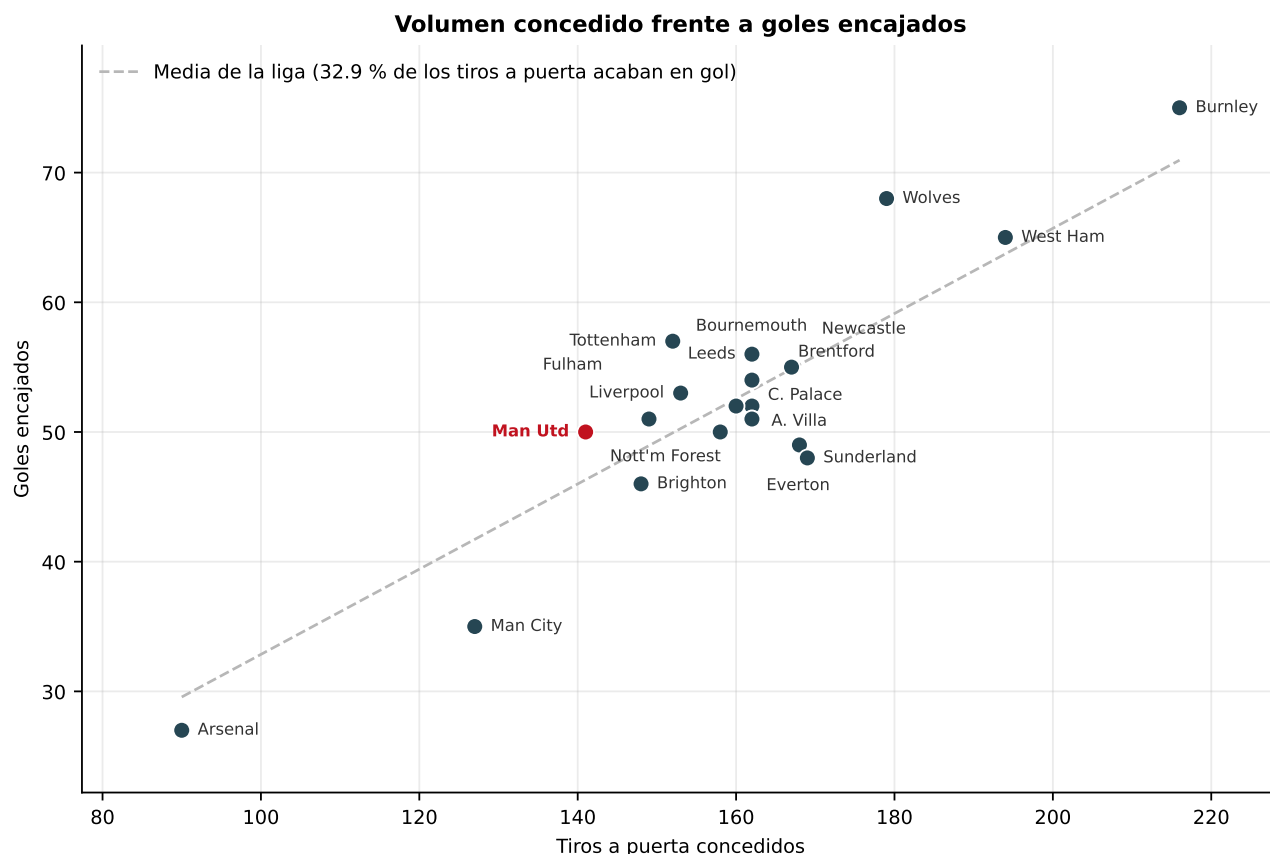


Figura 3: Tiros a puerta concedidos frente a goles encajados. Los equipos por encima de la línea encajan más de lo que su volumen concedido haría esperar.

Definición 3.1.

```
gc_esperados = utd["shots_on_target_contra"] * tasa_def
print(f"El United concedió sólo **{int(utd['shots_on_target_contra'])} tiros a "
      f"puerta**, el tercer registro más bajo de la liga tras Arsenal (89) y City "
      f"(126). Por ese volumen le habrían correspondido **{gc_esperados:.0f} goles "
      f"en contra**; encajó **{int(utd['goals_against'])}**, "
      f"**{utd['goals_against'] - gc_esperados:+.0f}** de más.\n\n"
      f"Con el porcentaje de paradas del City (72,2%) en lugar del suyo "
      f"({utd['gk_save_pct']:.1f}%), el United habría encajado unos "
      f"{utd['shots_on_target_contra'] * (1 - 0.722):.0f} goles en vez de 50.")
```

El United concedió sólo **141 tiros a puerta**, el tercer registro más bajo de la liga tras Arsenal (89) y City (126). Por ese volumen le habrían correspondido **46 goles en contra**; encajó **50**, **+4** de más.

Con el porcentaje de paradas del City (72,2%) en lugar del suyo (64.8%), el United habría encajado unos 39 goles en vez de 50.

El bloque defensivo funcionó —muy pocos tiros a puerta concedidos—, pero la portería no sostuvo ese trabajo. **Es el desajuste más grande del equipo**: tercera mejor defensa por volumen concedido y sólo sexta por goles encajados, con el puesto 16 de 20 en porcentaje de paradas.

```

d = liga.sort_values("gk_save_pct")
fig, ax = plt.subplots(figsize=(7, 5))
ax.barh(d["team"], d["gk_save_pct"], color=resaltar(d["team"]))
ax.set_xlabel("Porcentaje de paradas")
ax.set_xlim(55, 75)
ax.xaxis.set_major_formatter(PercentFormatter())
ax.set_title("Porcentaje de paradas - Premier League 2025/26")
ax.grid(axis="y", visible=False)
for i, (_, r) in enumerate(d.iterrows()):
    ax.annotate(f"{r['gk_save_pct']:.1f}", (r["gk_save_pct"], i),
                xytext=(3, 0), textcoords="offset points", va="center", fontsize=7.5)
fig.tight_layout()
plt.show()

```

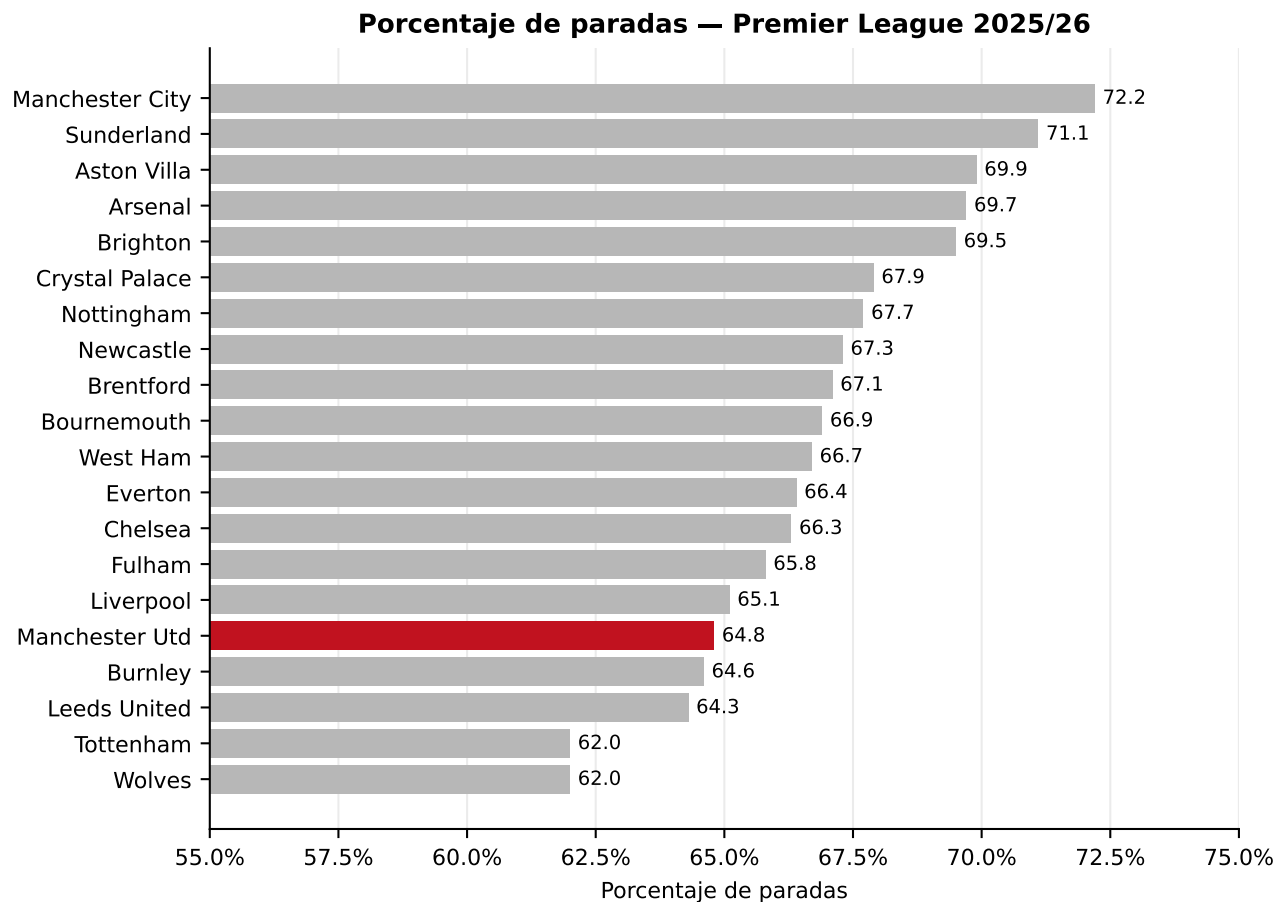


Figura 4: Porcentaje de paradas por equipo. El United queda en el puesto 16 de 20 pese a ser tercero en la clasificación.

4. La temporada partido a partido

```
pl = (united_partidos.query("comp == 'Premier League'")
      .sort_values("date")
      .reset_index(drop=True))
pl["jornada"] = pl.index + 1
pl["puntos"] = pl["result"].map({"W": 3, "D": 1, "L": 0})
pl["puntos_acum"] = pl["puntos"].cumsum()
pl["dif_acum"] = (pl["goals_for"] - pl["goals_against"]).cumsum()
pl["forma_5"] = pl["puntos"].rolling(5).mean()

# El cambio de sistema: 3-4-3 hasta la jornada 18, 4-2-3-1 a partir de ahí
pl["sistema"] = np.where(pl["jornada"] <= 17, "3-4-3", "4-2-3-1 (mayoritario)")
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 5.4), sharex=True,
                               gridspec_kw={"height_ratios": [2, 1]})

colores = {"W": VERDE, "D": AMBAR, "L": ROJO}
ax1.plot(pl["jornada"], pl["puntos_acum"], color=AZUL, linewidth=1.6, zorder=2)
ax1.scatter(pl["jornada"], pl["puntos_acum"],
            c=[colores[r] for r in pl["result"]], s=32, zorder=3,
            edgecolor="white", linewidth=0.6)
ax1.set_ylabel("Puntos acumulados")
ax1.set_title("Manchester United - trayectoria en la Premier League 2025/26")
ax1.axvline(17.5, color=GRIS, linestyle="--", linewidth=1)
ax1.annotate("cambio a 4-2-3-1", (17.5, 8), xytext=(4, 0),
            textcoords="offset points", fontsize=7.5, color="#555")
for res, col in colores.items():
    ax1.scatter([], [], c=col, label={"W": "Victoria", "D": "Empate", "L": "Derrota"}[res], s=32)
ax1.legend(frameon=False, loc="upper left", ncol=3, fontsize=7.5)

ax2.plot(pl["jornada"], pl["forma_5"], color=ROJO, linewidth=1.6)
ax2.axhline(pl["puntos"].mean(), color=GRIS, linestyle=":", linewidth=1)
ax2.axvline(17.5, color=GRIS, linestyle="--", linewidth=1)
ax2.set_ylabel("Puntos/partido\n(media móvil 5)")
ax2.set_xlabel("Jornada")
ax2.set_ylim(0, 3)

fig.tight_layout()
plt.show()
```

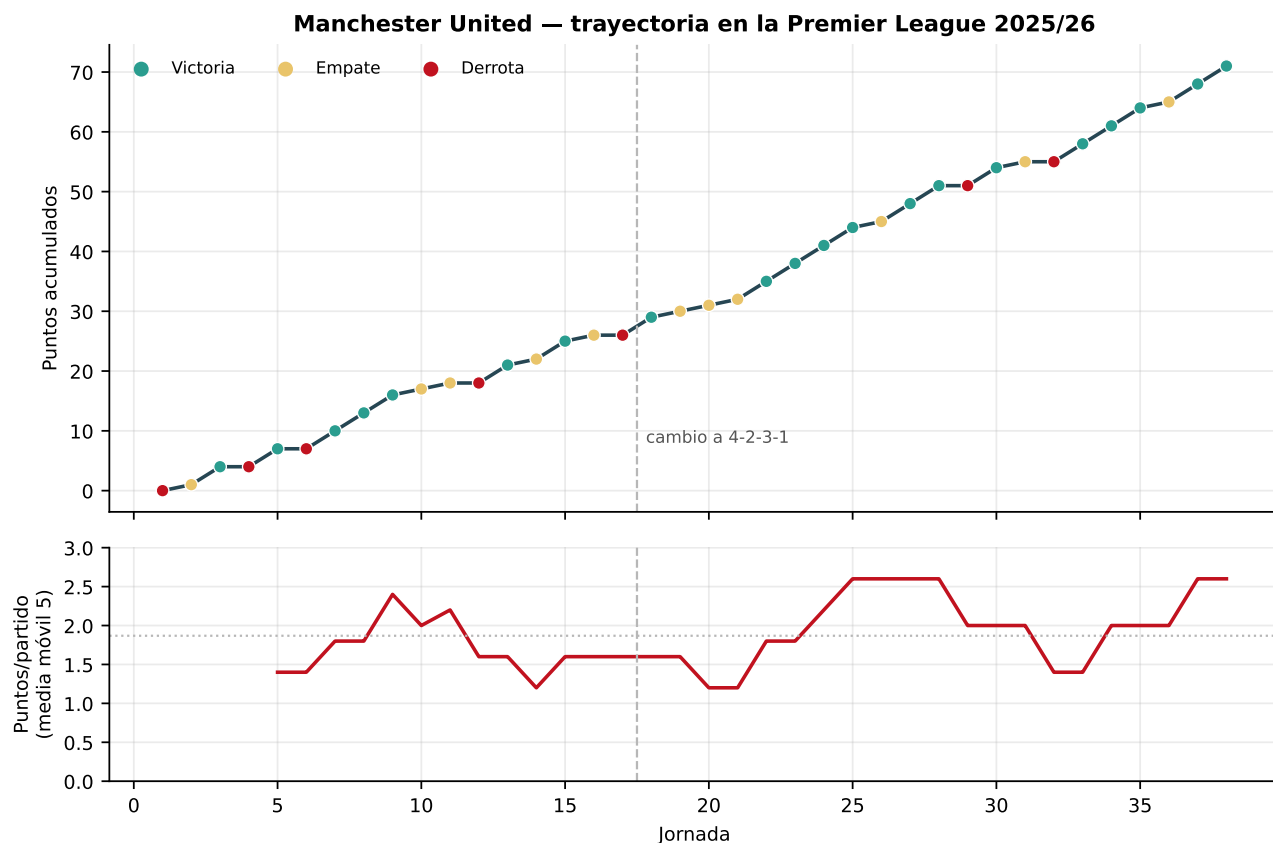


Figura 5: Puntos acumulados y forma en ventana de cinco partidos. La línea vertical marca el cambio de sistema tras la jornada 17.

Tabla 4: Rendimiento antes y después del cambio de dibujo tras la jornada 17.

Sistema	PJ	Pts	Pts/partido	GF	GC	Posesión media
3-4-3	17	26	1.53	31	28	54.59
4-2-3-1 (mayoritario)	21	45	2.14	38	22	49.76

El equipo cambió de 3-4-3 a 4-2-3-1 en la jornada 18 (26 de diciembre, 1-0 al Newcastle) y el rendimiento cambió con él. **Merece comprobarse si el cambio es causa o coincidencia:** el calendario de la segunda vuelta y la recuperación de efectivos son explicaciones alternativas que estos datos no descartan.

Tabla 5: Rendimiento como local y como visitante.

Condición	PJ	Pts	Pts/partido	GF	GC	Posesión media
Visitante	19	29	1.53	30	26	52.47
Local	19	42	2.21	39	24	51.37

5. Plantilla completa

Ahora sí, los 27 jugadores con minutos, en lugar del top 3 por categoría.

Tabla 6: Plantilla del United en Premier League 2025/26 (jugadores con al menos 90 minutos).

Jugador	Pos	Edad	PJ	Tit	Min	G	A	TA	TR	G/90	A/90
Luke Shaw	DF	30	38	38	3219	1	1	9	0	0.03	0.03
Bruno Fernandes	MF	30	35	35	3063	9	21	5	0	0.26	0.62
Senne Lammens	GK	23	32	32	2880	0	0	0	0	0.00	0.00
Diogo Dalot	DFMF	26	34	29	2614	1	3	5	0	0.03	0.10
Bryan Mbeumo	MFFW	25	33	31	2609	11	3	4	0	0.38	0.10
Casemiro	MF	33	34	33	2571	9	2	11	1	0.32	0.07
Matheus Cunha	MFFW	26	33	29	2494	10	2	4	0	0.36	0.07
Amad Diallo	MF	23	32	27	2341	2	3	2	0	0.08	0.12
Leny Yoro	DF	19	32	18	1740	0	0	1	0	0.00	0.00
Kobbie Mainoo	MF	20	28	16	1662	1	2	2	0	0.05	0.11
Harry Maguire	DF	32	23	19	1653	1	2	3	1	0.05	0.11
Benjamin Šeško	FW	22	30	17	1643	11	1	2	0	0.60	0.05
Patrick Dorgu	MF	20	26	15	1449	4	4	5	0	0.25	0.25
Lisandro Martínez	DF	27	18	13	1233	0	0	0	1	0.00	0.00
Matthijs de Ligt	DF	25	13	13	1170	1	0	0	0	0.08	0.00
Mason Mount	MF	26	23	12	1017	3	0	2	0	0.27	0.00
Noussair Mazraoui	DFMF	27	20	11	985	0	0	3	0	0.00	0.00
Ayden Heaven	DF	18	17	11	924	0	1	2	0	0.00	0.10
Manuel Ugarte	MF	24	22	8	887	0	0	1	0	0.00	0.00
Joshua Zirkzee	FWMF	24	24	5	626	2	1	3	0	0.29	0.14
Altay Bayındır	GK	27	6	6	540	0	0	0	0	0.00	0.00
Jack Fletcher	MF	18	3	0	108	0	0	0	0	0.00	0.00

5.1. Reparto de minutos

Tabla 7: Uso de la plantilla. «Completo» son partidos jugados de principio a fin.

Jugador	Pos	PJ	Titular	Comple- tos	Desde banquillo	Min	% de minutos	Pts/par- tido	+/-
Luke Shaw	DF	38	38	23	0	3219	94.10	1.87	21
Bruno Fernandes	MF	35	35	29	0	3063	89.60	1.89	20

Jugador	Pos	PJ	Titular	Comple- tos	Desde banquillo	Min	% de minutos	Pts/par- tido	+/-
Senne Lammens	GK	32	32	32	0	2880	84.20	2.00	23
Diogo Dalot	DFMF	34	29	22	5	2614	76.40	1.88	18
Bryan Mbeumo	MFFW	33	31	13	2	2609	76.30	1.97	21
Casemiro	MF	34	33	13	1	2571	75.20	1.97	24
Matheus Cunha	MFFW	33	29	16	4	2494	72.90	1.88	13
Amad Diallo	MF	32	27	17	5	2341	68.50	2.03	19
Leny Yoro	DF	32	18	9	14	1740	50.90	1.72	-5
Kobbie Mainoo	MF	28	16	14	12	1662	48.60	2.21	16
Harry Maguire	DF	23	19	13	4	1653	48.30	2.04	17
Benjamin Šeško	FW	30	17	8	13	1643	48.00	1.80	7
Patrick Dorgu	MF	26	15	10	11	1449	42.40	1.73	2
Lisandro Martínez	DF	18	13	11	5	1233	36.10	1.89	10
Matthijs de Ligt	DF	13	13	13	0	1170	34.20	1.62	1
Mason Mount	MF	23	12	3	11	1017	29.70	1.87	4
Noussair Mazraoui	DFMF	20	11	3	9	985	28.80	2.00	9
Ayden Heaven	DF	17	11	7	6	924	27.00	2.00	5
Manuel Ugarte	MF	22	8	3	14	887	25.90	1.55	-11
Joshua Zirkzee	FWMF	24	5	2	19	626	18.30	1.79	1
Altay Bayındır	GK	6	6	6	0	540	15.80	1.17	-4

💡 Sí: Luke Shaw jugó los 38 partidos

El dato llama la atención por su historial de lesiones, pero FBref es internamente consistente y admite comprobación: **38 partidos, 38 titularidades, 3 219 minutos**, que son el **94,1 %** de los 3 420 posibles. La cifra encaja porque no jugó los 38 completos, sólo **23**: fue sustituido en los otros 15, promediando 85 minutos por partido. No entró ni una vez desde el banquillo y no figura como suplente sin utilizar en ninguna jornada.

Fue el único jugador de la plantilla presente en los 38 partidos de liga. El segundo en minutos es Bruno Fernandes (3 063 en 35 partidos), que sí se perdió tres.

5.2. Producción ajustada por minutos

```
d = united_plantilla.query("minutes_90s >= 5 and position != 'GK']").copy()
# Etiqueta corta: apellido, con las excepciones donde el apellido solo no basta.
EXCEPCIONES = {"Matthijs de Ligt": "De Ligt", "Bruno Fernandes": "B. Fernandes",
               "Lisandro Martínez": "L. Martínez", "Amad Diallo": "Amad"}
d["etiqueta"] = [EXCEPCIONES.get(p, p.split()[-1]) for p in d["player"]]

fig, ax = plt.subplots(figsize=(8, 5.4))
ax.scatter(d["goals_per90"], d["assists_per90"],
          s=d["minutes_90s"] * 7, alpha=0.55, color=AZUL,
          edgecolor="white", linewidth=0.8, zorder=3)

# Se etiqueta a todos, empezando por los de mayor producción total para que sean
# ellos quienes se queden con el hueco si hay disputa.
d = d.sort_values("goals_assists_per90", ascending=False)
etiquetar(ax, d["goals_per90"].values, d["assists_per90"].values,
          d["etiqueta"].tolist(), fontsize=7.5, dist=9)

ax.set_xlabel("Goles por 90 minutos")
ax.set_ylabel("Asistencias por 90 minutos")
ax.set_title("Producción ofensiva ajustada por minutos (mínimo 5 partidos completos)")
ax.margins(x=0.14, y=0.16)
fig.tight_layout()
plt.show()
```

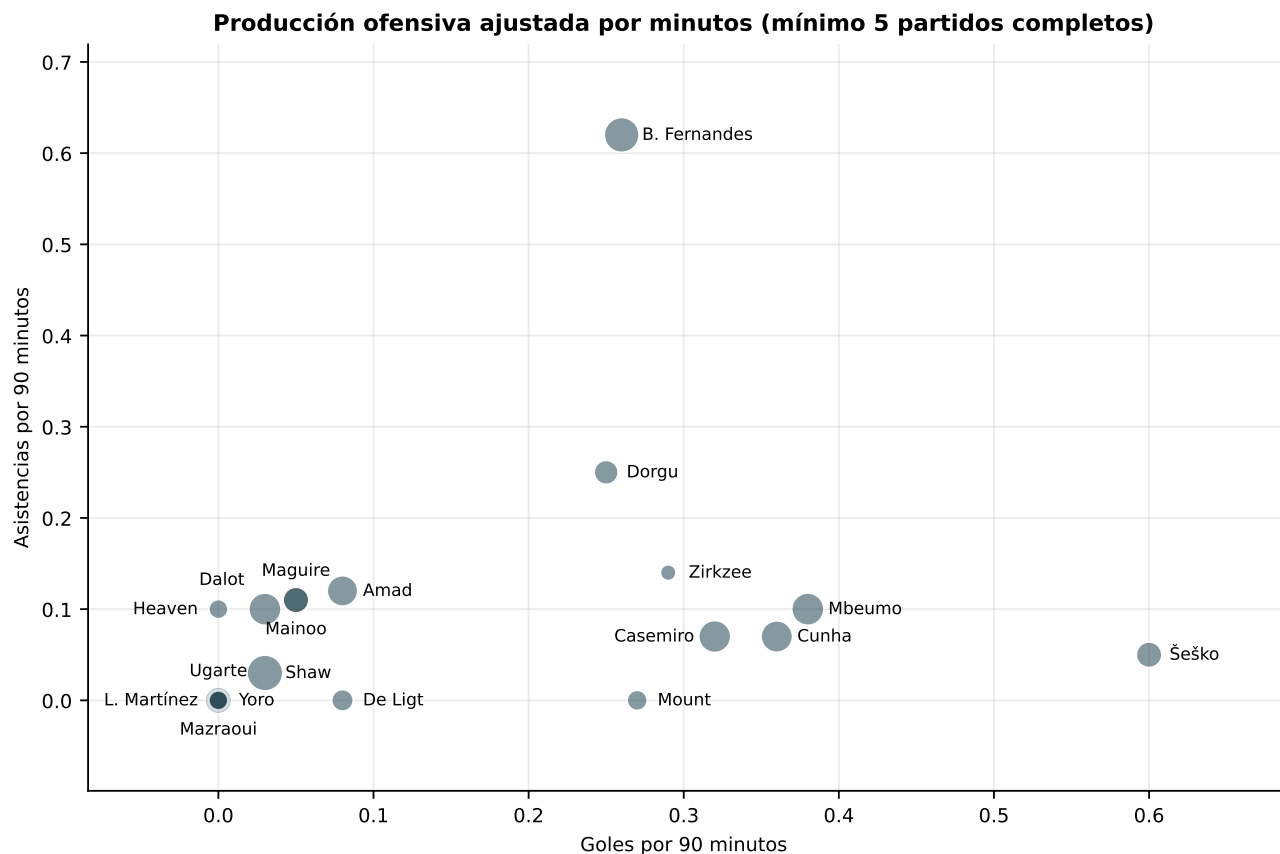



Figura 6: Goles y asistencias por 90 minutos. El tamaño del punto es proporcional a los minutos jugados.

```
s = united_plantilla.set_index("player")
sesko, mbeumo = s.loc["Benjamin Šeško"], s.loc["Bryan Mbeumo"]
print(f"**Šeško y Mbeumo empataron a 11 goles con cargas muy distintas.** "
      f"Šeško los marcó en {sesko['minutes']:,.0f} minutos "
      f"({sesko['games_starts']:,.0f} titularidades) y Mbeumo en "
      f"{mbeumo['minutes']:,.0f} ({mbeumo['games_starts']:,.0f}). "
      f"Por 90 minutos: **{sesko['goals_per90']:,.2f} contra "
      f"{mbeumo['goals_per90']:,.2f}**, una diferencia del "
      f"{(sesko['goals_per90']/mbeumo['goals_per90']-1)*100:.0f}% a favor de Šeško, "
      f"que fue además el jugador más eficiente de la plantilla.")
```

Šeško y Mbeumo empataron a 11 goles con cargas muy distintas. Šeško los marcó en 1,643 minutos (17 titularidades) y Mbeumo en 2,609 (31). Por 90 minutos: **0.60 contra 0.38**, una diferencia del 58 % a favor de Šeško, que fue además el jugador más eficiente de la plantilla.

Con los datos de Marca esta diferencia era invisible: la ficha daba medias de 0,37 y 0,33, sugiriendo que Šeško había jugado apenas tres partidos menos. En realidad jugó **casi mil minutos menos** y fue, con diferencia, el delantero más rentable por minuto del equipo.

5.3. Bruno Fernandes, otra vez el eje

Tabla 8: Peso de Bruno Fernandes en la producción del equipo.

Métrica	Bruno	Equipo	% del equipo
Asistencias	21	46	45.65
Centros	181	639	28.33
Goles	9	66	13.64
Penaltis lanzados	6	6	100.00

Jugó 3 063 minutos —el segundo del equipo tras Luke Shaw, que disputó los 38 partidos completos— y firmó **21 de las 46 asistencias** y **181 de los 639 centros**. La dependencia es real y cuantificable.

6. Síntesis

Tabla 9: El United en una tabla.

Indicador	Valor
Posición final	3.º de 20
Puntos	71
Diferencia de goles	+19
Tiros a puerta	215 — 1.º de la liga
Precisión de tiro	36.1 % — 1.º de la liga
Goles por tiro a puerta	0.29 — 9.º de la liga
Tiros a puerta concedidos	141 — 3.º mejor
Porcentaje de paradas	64.8 % — 16.º de 20
Porterías a cero	8 — 15.º de 20
Posesión	51.9 % — 8.º de la liga

La temporada se sostiene sobre una asimetría clara: **el United fue el mejor equipo de la liga generando remate franco y uno de los peores aprovechándolo, tanto en ataque como bajo palos**. Terminó tercero en la clasificación con el mayor volumen de tiros a puerta de la Premier y el puesto 16 de 20 en porcentaje de paradas.

Es un perfil poco común y con una implicación directa: el margen de mejora no está en crear más ocasiones —ya lidera— sino en definir las y en la portería.

7. Líneas de análisis abiertas

Todos los datos necesarios para las siete primeras ya están cargados.

Tabla 10: Candidatos para las siguientes secciones.

#	Análisis	Datos	Qué respondería
1	Perfil del United contra la media de la liga (z-scores por métrica)	Listos	En qué es realmente atípico
2	Ranking de eficiencia ofensiva y defensiva de los 20 equipos	Listos	Si el desajuste del United es único
3	Rendimiento contra rivales del <i>top 6</i> frente al resto	Listos	Si compitió contra los grandes
4	Efecto del cambio de sistema, controlando por dificultad de rival	Listos	Si el 4-2-3-1 explica la segunda vuelta
5	Comparación directa con City y Arsenal, métrica a métrica	Listos	Qué le faltó para pelear el título
6	Reparto de minutos y dependencia de titulares	Listos	Fondo de armario y rotación
7	Relación entre posesión y resultado, partido a partido	Listos	Si le convino tener el balón
8	xG contra goles reales	Requiere Understat o FBref avanzado	Si la mala definición fue mala suerte
9	Comparación con temporadas anteriores del United	Requiere descargar 2019/20–2024/25	Magnitud real de la mejora

i Reproducibilidad

Los CSV de `datos/` se generaron desde FBref el 16 de agosto de 2026. El script `scripts/descargar_fbref.py` automatiza la descarga, aunque FBref responde 403 a peticiones directas desde este entorno: los datos actuales se extrajeron del DOM con el navegador. Todas las cifras derivadas se calculan en este documento.